AI Engineering and Claude Architect · Part I, Foundations

AI and ML Fundamentals, Lesson 2

Classical ML and How to Measure It

A fraud model reaches 97.9% accuracy. So does a single line of code that says "never fraud". Learning to tell those two apart is the most useful skill in this module, and it carries straight through to measuring Claude.

Module	AI01 of 13
Lesson	2 of 4
Prerequisites	AI01 L1
Exam relevance	Algorithms are out of scope. Measurement thinking supports TS 4.1 (false positives) and TS 5.5 (aggregate metrics hide weak segments)
Reading time	~45 minutes

Contents

1. Classical ML and How to Measure It

Lesson 2 of 4 · AI and ML Fundamentals

Classical ML and How to Measure It

Key idea

A model is only as trustworthy as the way you measured it. Classical algorithms differ in how they draw boundaries, and for tabular data they remain excellent. But the choice of metric matters more than the choice of algorithm. **Accuracy can look superb on a useless model, an average can hide a segment that's completely broken, and a score on training data proves nothing.** Use held-out data, precision and recall, and per-segment results. You'll use all three again to evaluate Claude.

> *Before: features, labels, parameters and generalization (AI01 L1). Teaches: the classical algorithms and when to use each, overfitting, data splits and leakage, precision, recall, F1, thresholds and segment analysis. Used by: AI05 L6 (retrieval metrics), AI10 L3 (evals), AI12 L5 (false positives in code review) and AI13.*

The hook: 97.9% accurate, catches nothing

A team reports a great result: their fraud model is 97.9% accurate. You look at the test set: 20,000 transactions, of which 412 are fraud. Then you try the simplest possible "model":

```
boolean isFraud(Txn txn) {
    return false;
}
```

Real output from `code/ai01-ml/FraudMetrics.java`, on synthetic data:

```
test set: 20000 transactions, 412 fraud
always say legitimate: accuracy=0.979 precision=0.000 recall=0.000 f1=0.000
```

Key idea

One line of code ties their accuracy and catches zero fraud. When one outcome is rare, accuracy mostly measures how rare it is. This is called the **accuracy paradox**. It's why a serious team never reports accuracy alone on imbalanced data.

Intuition: an airport scanner

An airport scanner can make two kinds of mistake. It can **raise a false alarm**, which annoys a traveler. Or it can **miss a real threat**, which is far worse. Saying "95% accurate" tells you nothing about which mistake it makes.

Every metric in this lesson is a way of asking which mistakes a model makes, and what those mistakes cost you.

The classical algorithms

Your ML handbook lists these. You won't implement them in Java at work, but you should know what each does and when you'd pick it. All except K-means and PCA are supervised.

Algorithm	Core idea	Good for	Weak at
Linear regression	Fit a straight line through numbers	Predicting a quantity, as a baseline	Non-linear patterns
Logistic regression	A weighted sum squashed into a probability	Explainable yes/no decisions	Feature interactions
Decision tree	A learned series of if/else questions	Explaining decisions	Overfits on its own
Random forest	Many varied trees vote	Reliable default for tabular data	Size; less explainable
Gradient boosting	Each new tree fixes the last trees' errors	Often the strongest on tabular data	Tuning; can overfit
AdaBoost	Reweights examples that were previously wrong	Simple boosting	Sensitive to noise
K-nearest neighbors	Copy the answer from similar examples	Small data, similarity problems	Slow at scale, needs scaling
Support vector machine	Find the widest gap between classes	Smaller, high-dimensional data	Large datasets
Naive Bayes	Combine per-feature probabilities	Fast text baseline	Assumes independent features
K-means	Group points around K centers	Customer segments	You must pick K; expects round clusters
PCA	Compress features into fewer dimensions	Visualization, noise reduction	New features are hard to interpret

The ones worth understanding more deeply

Logistic regression matters most here, because a neural network is built from this same unit (AI01 L3). It computes a weighted sum and squashes it into a value between 0 and 1 using the **sigmoid** function:

`z = w₀ + w₁x₁ + w₂x₂`, then `p = 1 / (1 + e^(-z))`

```
static double predict(double[] w, double[] x) {
    return sigmoid(w[0] + w[1] * x[0] + w[2] * x[1]);
}
```

Each weight says how strongly a feature pushes towards "fraud", which makes the model easy to explain to an auditor. Training adjusts the weights by a small amount on each pass to reduce error. AI01 L3 explains exactly how.

Decision trees choose the question that best separates the classes. A common measure is **Gini impurity**: $1 - \Sigma\ p_i^2$. A node holding 8 fraud and 2 legitimate cases scores $1 - (0.8^2 + 0.2^2) = 0.32$. A 50/50 node scores 0.5, the worst possible with two classes, and a pure node scores 0. Training picks splits that lower this the most.

Ensembles answer the weakness of single trees, and the two main families work differently:

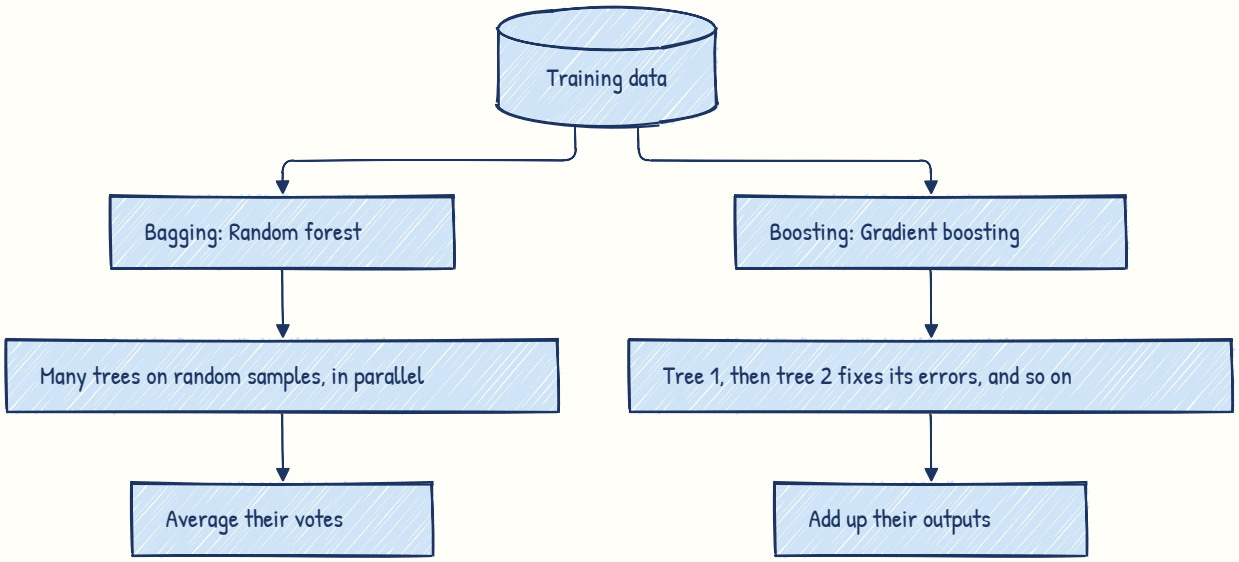


Figure 1. Two ways to combine trees. Bagging averages independent trees to reduce instability. Boosting chains them so that each one corrects the errors before it.

Why would many mediocre trees beat one good one? Because their errors differ, and averaging cancels errors that aren't shared. That idea returns in LLM work as running independent review passes, though with a specific caveat discussed in AI12 L5.

K-nearest neighbors stores the training data and copies from the closest examples. It's worth remembering because it's the same idea as vector search (AI04): find the nearest stored things.

What practitioners usually do with tabular data

Start with logistic regression as a baseline, then try gradient boosting (XGBoost or LightGBM are common libraries). This is an engineering recommendation. Deep learning's main advantage is on unstructured data like language and images. **For a table of transaction features, a neural network often isn't the first thing to try.**

Overfitting: the model that memorized the exam

Here's the failure every data-split rule exists to prevent. K-nearest neighbors with `k = 1` answers by copying the single closest training example. Tested on its own training data, each point finds itself. Real output, on noisy synthetic data with balanced classes:

```
knn k=1   train accuracy=1.000 test accuracy=0.648
knn k=15  train accuracy=0.740 test accuracy=0.678
knn k=101 train accuracy=0.725 test accuracy=0.700
```

Key idea

100% on the training data, 64.8% on new data. The `k=1` model memorized noise. The gap between training and test scores is the signature of **overfitting**. A score on data the model has already seen tells you almost nothing about its future performance.

Notice `k=101`: lower training score, higher test score. Simpler often generalizes better. The tension has a name:

	Underfitting (high bias)	Good fit	Overfitting (high variance)
Train score	Low	Good	Very high
Test score	Low	Good	Much lower than train
Cause	Too simple	Right complexity	Too complex, or too little data
Fix	Better features, richer model	Keep it	More data, simpler model, regularization (AI01 L3)

The three splits

Split	Purpose	Touched how often
Training	Fits parameters	Constantly
Validation	Chooses hyperparameters and compares models	Many times
Test	Gives one honest final estimate	Ideally once

Every time you check a set and adjust, it leaks a little information into your choices. A test set you've consulted 50 times has become a validation set. **Cross-validation** rotates the validation slice through the data, which helps when data is scarce.

This has a direct LLM counterpart. If you tune a prompt until it passes every case in your eval set, you may have overfit the prompt to those cases. AI10 L3 uses a held-out set for that reason.

Leakage: the bug that makes bad models look brilliant

Data leakage means training on information that wouldn't exist at prediction time. It produces wonderful test scores and terrible production results.

Leak	Why it's wrong	Fix
A `chargeback_filed` feature	Only exists after fraud is known	Use only what's known at decision time
Random split of time-ordered data	Trains on the future to predict the past	Split by time
Scaling using all rows, including test	Test statistics leak into training	Fit the scaler on training data only
Same customer in train and test	Model recognizes the customer, not the pattern	Split by customer

> *Watch out*
>
> **When a result looks too good, assume leakage until you've ruled it out.** It's among the most common causes of a great offline score that collapses in production. HLD M21C covers the related problem of training-serving skew.

Measuring what matters

The confusion matrix

Every metric comes from these four counts:

	Actually fraud	Actually legitimate
Flagged	True positive (TP)	False positive (FP): a false alarm
Not flagged	False negative (FN): a miss	True negative (TN)

Metric	Formula	The question it answers
Accuracy	(TP + TN) / all	How often is it right overall?
Precision	TP / (TP + FP)	When it flags something, how often is it right?
Recall	TP / (TP + FN)	Of all real fraud, how much did it find?
F1	2 × P × R / (P + R)	One number balancing both

```
static Scores of(int tp, int fp, int fn, int tn) {
    double precision = tp + fp == 0 ? 0 : (double) tp / (tp + fp);
    double recall = tp + fn == 0 ? 0 : (double) tp / (tp + fn);
    double f1 = precision + recall == 0 ? 0
            : 2 * precision * recall / (precision + recall);
    double accuracy = (double) (tp + tn) / (tp + fp + fn + tn);
    return new Scores(accuracy, precision, recall, f1);
}
```

Thresholds: where the business decision hides

Logistic regression outputs a probability. You choose the cutoff, and that choice trades precision against recall. Real output from the same model at three thresholds:

```
threshold 0.50: accuracy=0.983 precision=0.776 recall=0.235 f1=0.361
threshold 0.20: accuracy=0.979 precision=0.496 recall=0.461 f1=0.478
threshold 0.05: accuracy=0.932 precision=0.196 recall=0.743 f1=0.310
```

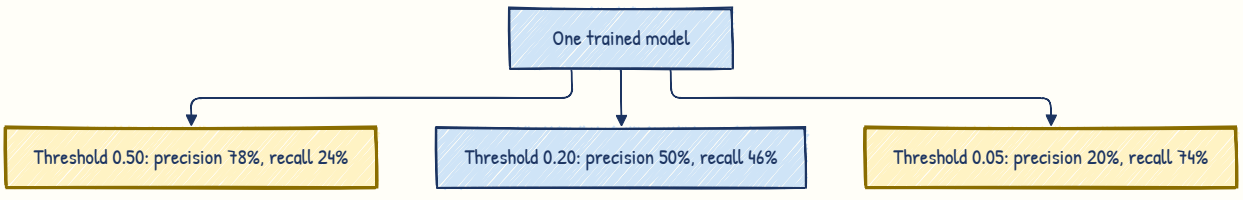


Figure 2. The same model at three thresholds. A lower threshold finds more fraud, and raises more false alarms. Figures are from the synthetic demo.

Accuracy would pick threshold 0.50, and that catches under a quarter of the fraud. F1 would pick 0.20. **But neither knows your costs.** So calculate them directly. Suppose a manual review costs $5, and an undetected fraud costs $200. Reviewing is worthwhile when:

`p × 200 > 5`, which gives `p > 0.025`

So a threshold near 0.025, far below 0.5, is the economically sensible choice under those assumptions, as long as your probabilities are reliable. This ignores customer friction, which you'd add as another cost.

> *Key idea*
>
> **A threshold is a business decision disguised as a technical parameter.** Ask what each kind of mistake costs, and derive it.

Domain	Worse mistake	Favor
Fraud screening followed by human review	Missing fraud	Recall
Auto-blocking a customer's card	Blocking a good customer	Precision
Automated PR review comments	Noisy false alarms that destroy trust	Precision
Finding relevant chunks in retrieval	Missing the right chunk	Recall at the retrieval stage

For comparing models across all thresholds, two summary curves are common. **ROC AUC** plots recall against the false positive rate. **Precision-recall AUC** is often more informative when positives are rare. Their official definitions are in the [scikit-learn evaluation guide]. For numeric prediction, use **MAE** (average error size) or **RMSE**, which penalizes large errors more.

The average that hides a disaster

So you've chosen 0.20. Overall recall is 46%. Now split the same results by payment channel:

```
segment card   (threshold 0.20): accuracy=0.981 precision=0.518 recall=0.524 f1=0.521
segment wallet (threshold 0.20): accuracy=0.966 precision=0.056 recall=0.020 f1=0.029
```

> *Watch out*
>
> **Wallet recall is 2%. The model is effectively blind there, and overall numbers didn't show it.** Wallets are about 10% of traffic, so they barely move the average. This is constructed: the synthetic wallet fraud carries no signal in these features. Real systems hit this in less obvious ways, and a fraud ring that discovers the blind spot will use it.

This is one of the most directly exam-relevant ideas in this module. Task Statement 5.5 says aggregate accuracy can mask poor performance on specific document types or fields, and requires checking segments before reducing human review.

Class imbalance

Technique	What it does	Watch out for
Better metrics	Precision, recall and PR AUC	Always do this first
Class weights	Makes rare-class errors cost more	Probabilities need recalibrating
Resampling	Under-sample the majority, or over-sample the rare class	Do it only on training data, never on test
Threshold tuning	Moves the operating point	Needs costs and a validation set

Why the obvious fixes fall short

"Fix" for a bad result	Why it falls short	Better
Report higher accuracy	Doesn't reflect rare events	Precision and recall
Try a fancier algorithm	The problem may be features, labels or leakage	Check the data first
Add more features	May add leakage	Check each one was known at decision time
Tune on the test set	Results become optimistic	Use a validation set
Report one global score	Hides weak segments	Report by segment

How the field solved it

These methods grew from statistics and applied ML over decades, not from one company. You can see the same discipline in Anthropic's public guidance: its evaluation guide asks for task-specific tests with edge cases, specific measurable criteria, and automated grading where possible.

> *In the real world*
>
> The exam guide applies these ideas directly to Claude. It warns that a false positive problem in one review category undermines confidence in accurate categories (TS 4.1), and that "97% overall" can mask poor performance (TS 5.5). So the concepts in this lesson are general ones. Only the algorithms are out of scope.

The production architecture

This is a recommended reference design, not a specific company's system. It shows how evaluation becomes a release gate.

Baseline: a notebook number. Someone runs a notebook, sees a good score, and ships.

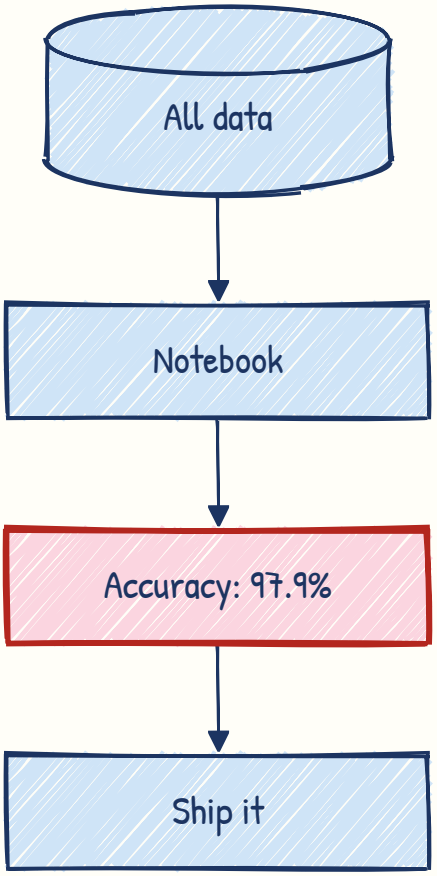


Figure 3. Baseline. A single accuracy number decides the release, and nobody can reproduce it later.

Improved: a repeatable offline evaluation. A fixed, time-split test set and the right metrics. New problem: **it can pass offline and still fail live**, because live data differs and the segment results aren't reviewed.

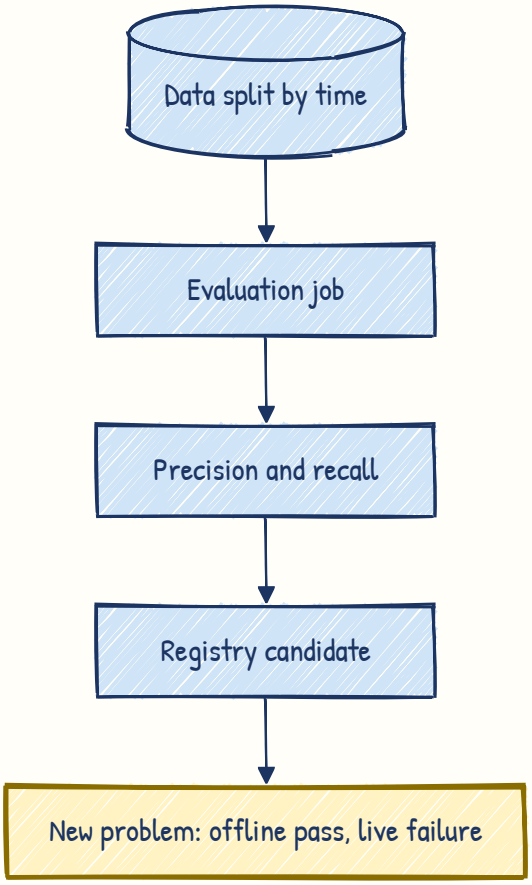


Figure 4. Improved. The metrics are now correct and repeatable, but offline results still don't prove live behavior.

Production: gated stages. Each stage can stop the release before customers feel it.

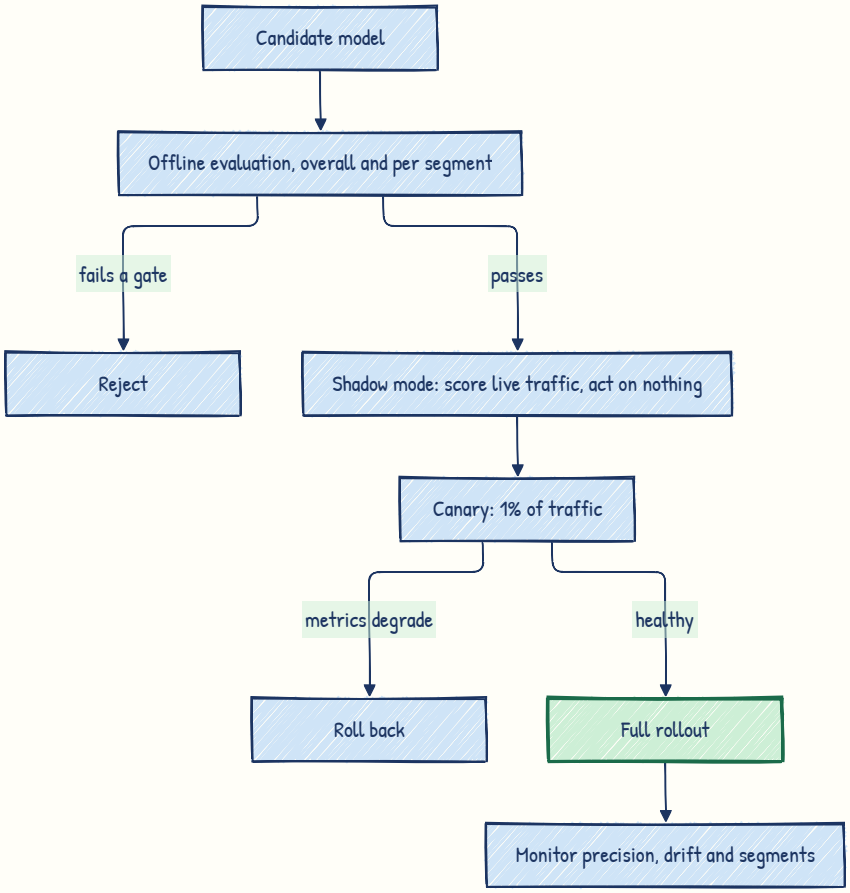


Figure 5. Production. Segment gates block a blind spot, shadow mode tests live traffic with no customer impact, and a canary limits the blast radius.

Component	Why it exists	How it communicates
Offline evaluation	Cheap, repeatable first check	CI or pipeline job reading a versioned test set
Segment gates	Stop a model that only looks good on average	Fail if any key segment falls below its floor
Shadow mode	Tests on real traffic with no risk	Payment service sends a copy of each request asynchronously
Canary	Limits how many customers a bad model reaches	Router splits traffic by percentage
Monitoring	Finds decay after launch	Metrics pipeline, plus delayed labels

You'll build this same ladder for prompts and models in AI10 L3: evals in CI, then shadow, then canary.

Java

Most Java teams don't train models. They serve them, or call them. Common options, as an engineering recommendation:

Need	Typical approach
Train classical models	Python, with the artifact exported
Serve in the JVM	ONNX Runtime has a Java API, and XGBoost has JVM bindings
Serve separately	A Python model service called over HTTP or gRPC
Evaluate	Compute metrics in the pipeline, using one shared definition

One thing Java teams should always own: the metric code and the decision thresholds. They hold business logic, so version them, test them, and don't bury them in a notebook.

Trade-offs

Decision	You gain	It costs
Logistic regression	Explainable, fast	Misses interactions without manual features
Gradient boosting	Often top accuracy on tables	Harder to explain, more to tune
Lower threshold	Higher recall	More false alarms and review cost
Higher threshold	Higher precision	More misses
Time-based splits	Honest estimates	Less training data, stale test sets
Segment gates	Catch blind spots	More rules to maintain; small segments give noisy numbers

When not to optimize further: if a simple baseline already meets the business goal, stop. Each point of accuracy costs more than the last.

Common mistakes

1. **Reporting accuracy on imbalanced data.** It rewards never finding anything.
2. **Evaluating on the training data.** That measures memory.
3. **Random splits on time-based data.** That trains on the future.
4. **Keeping the default 0.5 threshold.** Nothing about your business makes 0.5 right.
5. **Trusting one global number.** A segment can be broken while the average looks fine.
6. **Believing a suspiciously good score.** Look for leakage first.
7. **Tuning against the test set until it passes.** It stops being a test.

What to say in an interview

If asked to evaluate a fraud model, say: "Accuracy isn't useful at a 2% fraud rate. I'd use precision and recall on a time-based holdout, derive the threshold from what each mistake costs, and check each segment. Then I'd run it in shadow mode and canary it." The segment and cost points are what separate a strong answer.

Certification lens

How this lesson maps to CCAR-F

Algorithms aren't tested. Two ideas are: **false positive rates damage trust** (TS 4.1), and **aggregate metrics can hide weak segments** (TS 5.5). AI10 L3 adds stratified sampling, and AI12 L5 applies false positive control to code review.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Structured Data Extraction · Hard. Your extraction pipeline reports 96% field accuracy overall, and management wants to remove human review. What should you do first?

A. Remove review for extractions with high model confidence, since 96% is above target.

B. Measure accuracy by document type and by field, using a labeled sample, before reducing review.

C. Retry each extraction until two runs agree.

D. Add a second model and average both confidence scores.

A **B.** Aggregate accuracy can mask poor performance on particular document types or fields (TS 5.5). A uses uncalibrated confidence and ignores segments. C adds cost, and agreement doesn't prove correctness. D also doesn't measure real error rates.

Q Q2 · Output prediction · Medium. A model gives TP = 40, FP = 10, FN = 60, TN = 890. Compute accuracy, precision and recall. What do they tell you?

A Accuracy = 930/1000 = 93%. Precision = 40/50 = 80%. Recall = 40/100 = 40%. So when it flags something, it's usually right, but it misses 60% of real cases. The 93% hides that.

Q Q3 · Troubleshooting · Hard. A churn model scores 0.99 AUC offline and does no better than guessing in production. A top feature is `cancellation_reason`. What happened?

A Data leakage. A cancellation reason exists only after a customer has churned, so it isn't available at prediction time. Remove features that aren't known at decision time, split by time, and re-evaluate. Treat any near-perfect score as suspect until you've checked this.

Revision - Lesson 2

→ Accuracy is misleading on rare events: "never fraud" scored 97.9% in the demo and caught nothing.

→ Precision: of what you flagged, how much was right. Recall: of what was real, how much you found.

→ A threshold trades precision against recall. Derive it from the costs of each mistake.

→ Overfitting: great on training data, poor on new data. k=1 scored 100% in training and 64.8% in test.

→ Training fits, validation chooses, test gives the final estimate.

→ Leakage means using information unavailable at prediction time. Too-good results are suspicious.

→ Report per segment. The demo's wallet recall was 2% and overall numbers hid it.

→ Bagging averages independent trees. Boosting chains trees to correct errors.

→ For tabular data, start with logistic regression, then try gradient boosting.

→ Release ladder: offline with segment gates, shadow, canary, then monitor.

Your notes